



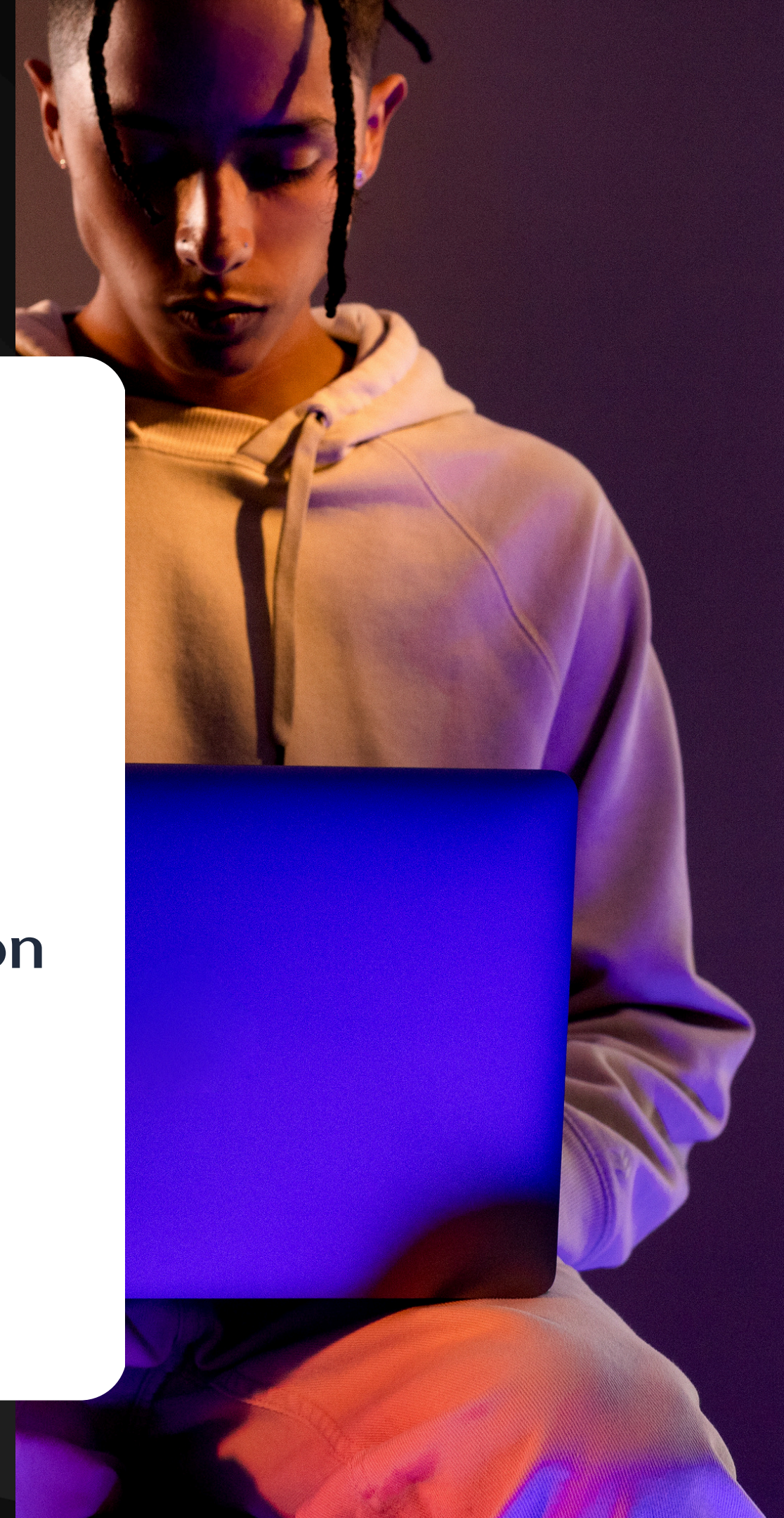
Secure Messaging Application



Presented By:
Nuradil Kanat
Didar Nurdaulet
Sherkhan Kudaibergen

Overview

1. Problem Statement
2. Architecture Overview
3. Key Exchange
4. Encryption & Integrity
5. Password Security
6. Authentication & Authorization
7. Message Security
8. Key Features
9. Challenges Faced & Solutions
10. Technical Highlights



Problem Statement

- 01 Traditional messaging platforms store messages in plaintext on servers
- 02 Users have no control over their private communications
- 03 Data breaches expose millions of messages to attackers
- 04 Weak authentication leads to account compromises
- 05 No forward secrecy – past messages remain vulnerable



Architecture Overview

Backend (Python/Fast API)

- Secure FAST API
- SQLite-based encrypted message storage
- Authentication, messaging, cryptography modules
- Http Only session cookies

Frontend (JavaScript SPA)

- Modern single-page interface
- Secure cookie authentication
- Local encryption/decryption
- MFA QR-code interface

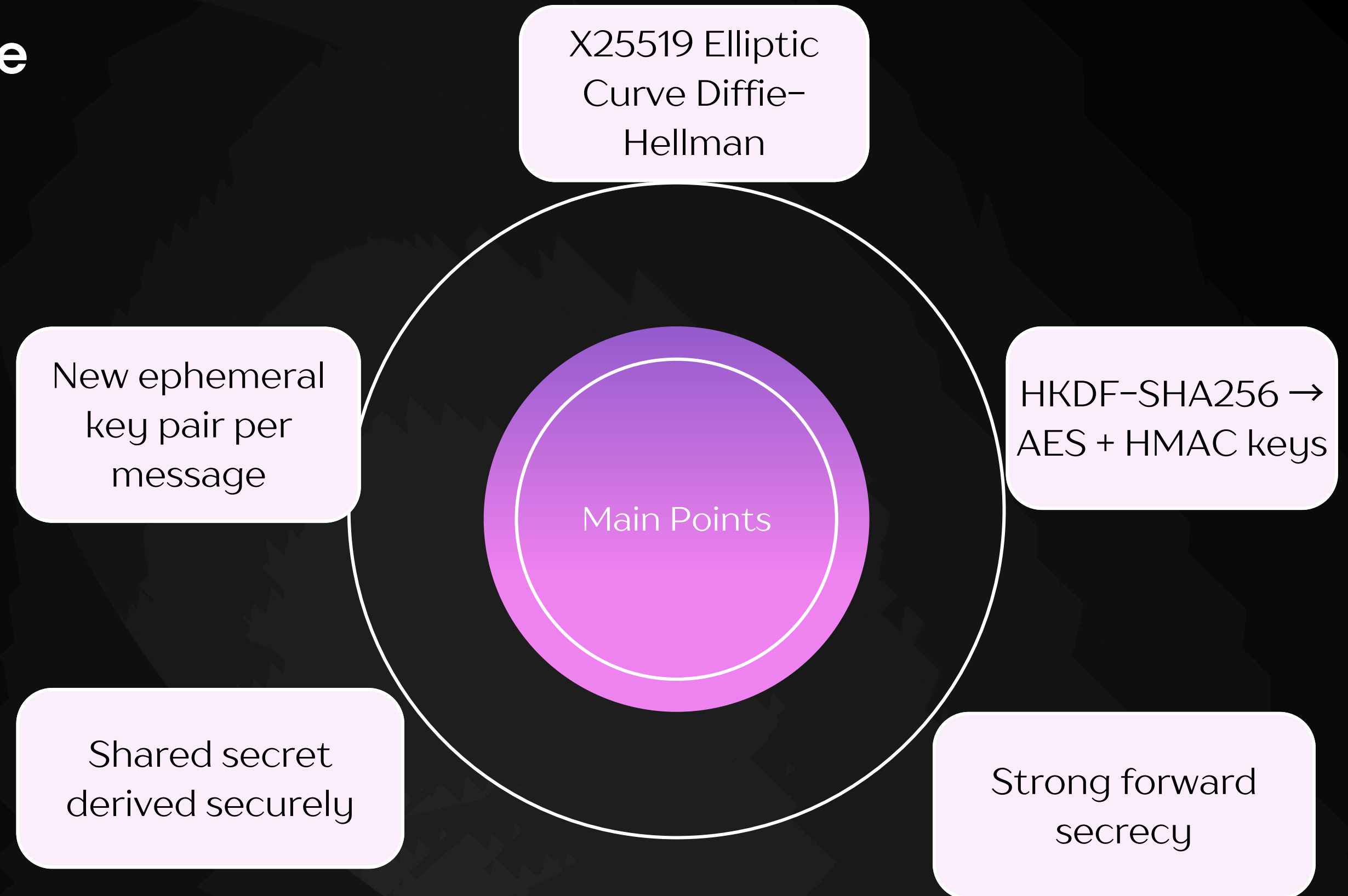
Key Components

crypto.py
app.py
server.py
db.py
totp.py
validation.py

Key Exchange

Why X25519:

1. Used by Signal, WhatsApp, Wire
2. Fast and efficient
3. Timing-attack resistant



Encryption & Integrity

AES-256-
GCM

- Strong symmetric encryption
- Unique nonce per message
- Built-in authentication tag

HMAC-
SHA256

- Extra integrity verification
- Protects against tampering

Ed25519
Signatures

- Verifies sender identity
- Signs encrypted message envelope

Password & Key Security

Bcrypt:

- Adaptive hashing
- Salted and brute-force resistant

PBKDF2:

- 200,000 iterations
- Encrypts private keys at rest

Password Requirements:

- 8+ characters
- Must include letter, number, symbol

Authentication & Session Security

Multi-Factor Authentication (TOTP)

- 1) RFC 6238
- 2) Works with Google/Microsoft Authenticator
- 3) Rolling 30-second codes

Secure Sessions

- 1) HttpOnly cookies
- 2) SameSite=Lax protection
- 3) 24-hour session lifetime

Password Reset

- 1) 32-byte secure tokens
- 2) 1-hour validity
- 3) Non-enumeration responses

Cryptography

AES-256 for Message Encryption

```
encrypted = crypto.encrypt_message(aes_key, plaintext.encode("utf-8"), hmac_key)
envelope = {
    "nonce": encrypted["nonce"],
    "ciphertext": encrypted["ciphertext"],
    "hmac": encrypted["hmac"],
}
```


Cryptography

ECDH (X25519) for Key Exchange

```
eph_private = x25519.X25519PrivateKey.generate()  
aes_key, hmac_key = crypto.derive_message_key(  
    eph_private, crypto.b64decode(recipient_public["x25519"])  
)
```

Cryptography

HMAC and Ed25519

Signatures

```
signature_payload = json.dumps(envelope, sort_keys=True).encode("utf-8")
signature = crypto.sign_message(session.secrets.ed25519_private, signature_payload)
envelope["signature"] = crypto.b64encode(signature)
```

```
signature_valid = crypto.verify_signature(
    crypto.b64decode(sender["public"]["ed25519"]),
    payload,
    crypto.b64decode(msg["signature"]),
)
```

Cryptography

Secure Password Hashing with bcrypt

```
password_hash = crypto.hash_password(password)
secrets = crypto.unwrap_private_keys(password, user["wrapped_keys"])
wrapped_keys = crypto.wrap_private_keys(password, secrets)
```

Cryptography

Secure Password Hashing with bcrypt

```
password_hash = crypto.hash_password(password)
secrets = crypto.unwrap_private_keys(password, user["wrapped_keys"])
wrapped_keys = crypto.wrap_private_keys(password, secrets)
```


Cryptography

TOTP (Time-based One-Time Password) for MFA

```
def generate_totp_secret(username: str) -> str:  
    return pyotp.random_base32()
```

```
def generate_totp_qr_code(secret: str, username: str, issuer: str = "Secure Messaging") -> str:  
    totp_uri = pyotp.totp.TOTP(secret).provisioning_uri(name=username, issuer_name=issuer)  
    # Generate QR code as base64 PNG  
    ...  
    return f"data:image/png;base64,{img_data}"
```

```
def verify_totp(secret: str, token: str, window: int = 1) -> bool:  
    totp = pyotp.TOTP(secret)  
    return totp.verify(token, valid_window=window)
```

Message Security

End-to-End Encryption	Forward Secrecy	Integrity Layers	Group Messaging
Server cannot read messages	New ephemeral key per message	AES-GCM	Per-recipient encryption
Full confidentiality	Past messages stay protected	HMAC-SHA256	Same security guarantees
Keys are generated and stored only on the user's device	Exposure of long-term keys does not compromise previous messages	Ed25519 signature verification	Each user receives an independently encrypted message package

Demo Overview

User Registration

- Username, password, email
- Automatic key generation
- Private keys encrypted with PBKDF2

Sending Messages

- Generate ephemeral key
- Encrypt with AES-GCM
- HMAC + signature
- Store encrypted message

MFA Setup

- Generate TOTP secret
- Scan QR code

Receiving Messages

- Verify signature
- Validate HMAC
- Decrypt and display

Challenges & Solutions

Challenge 1: Forward Secrecy
Solution: Ephemeral X25519 keys

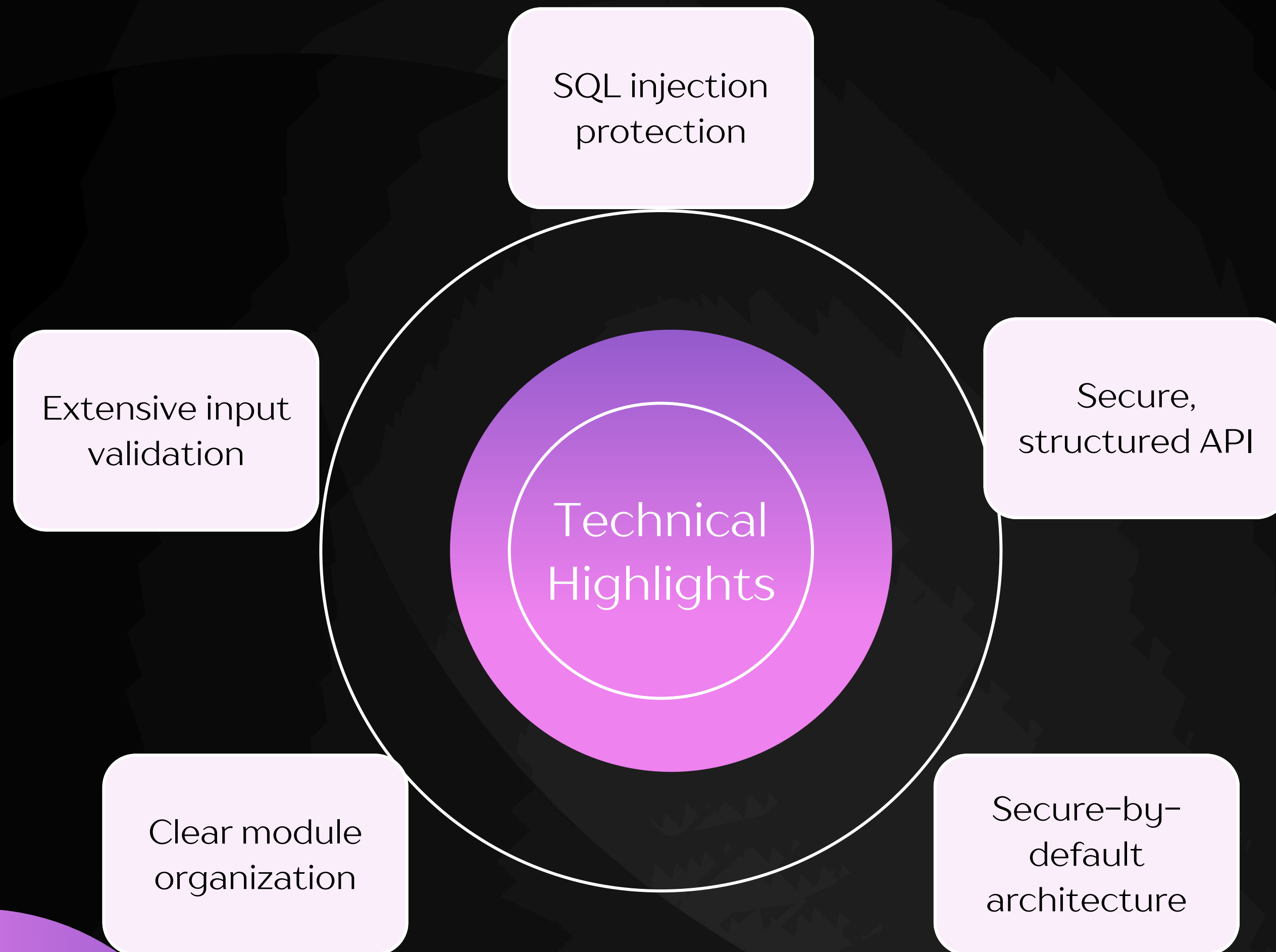
Challenge 2: Key Management
Solution: PBKDF2 + AES-GCM for private keys

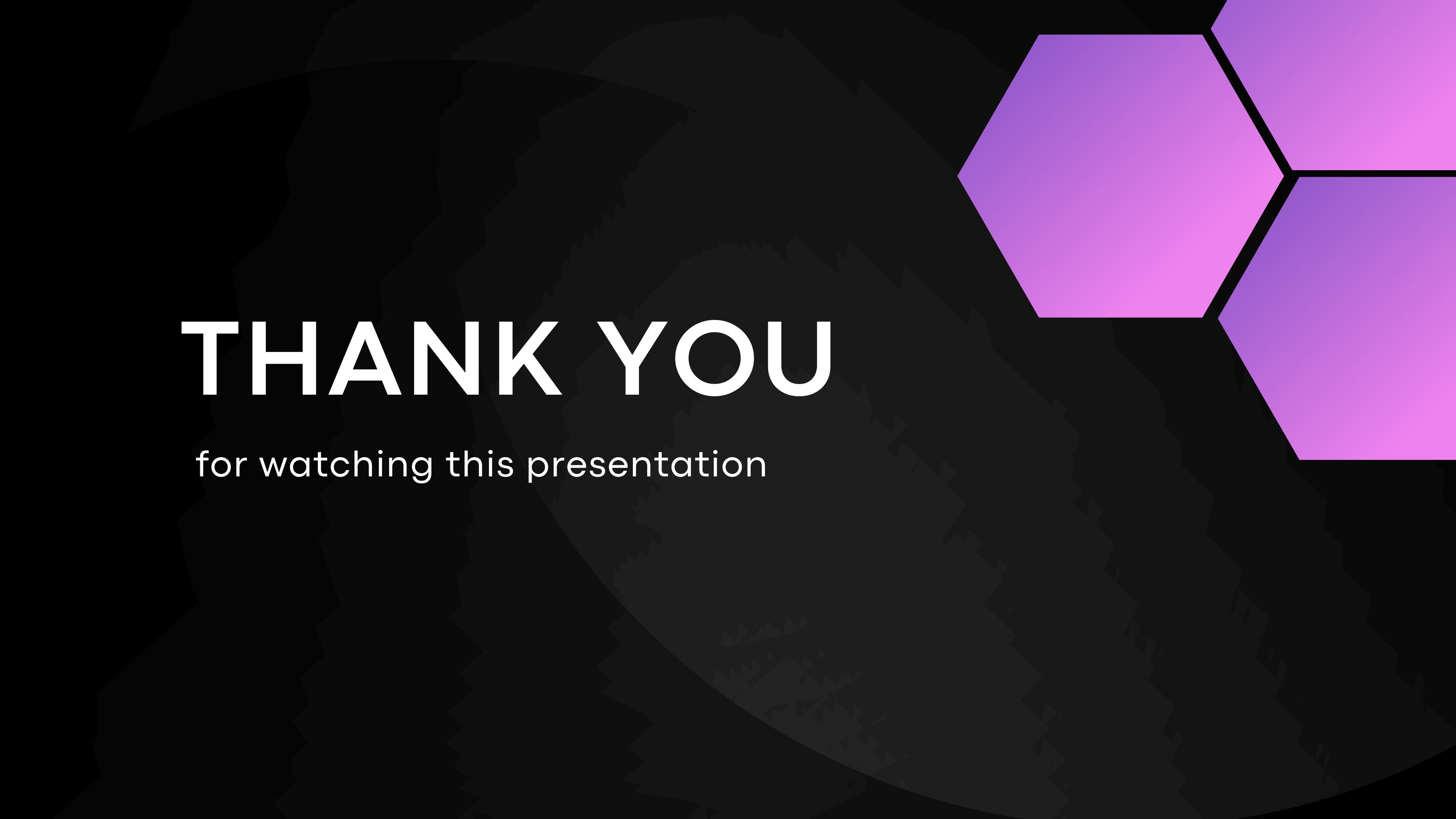
Challenge 3: Compatibility
Solution: Dual message format support

Challenge 4: Session Security
Solution: HttpOnly + Secure cookies

Challenge 5: Password Reset
Solution: Strong tokens & generic responses

Challenge 6: Timing Attacks
Solution: Constant-time comparisons



The background features a dark purple, textured sphere on the left side. On the right side, there are three pink hexagons arranged in a cluster, with one hexagon in the top right and two below it, slightly overlapping. The text "THANK YOU" is written in a large, white, sans-serif font, centered horizontally.

THANK YOU

for watching this presentation